

Software Test Plan (STP) – Ambulo App

1. Introduction

This Software Test Plan (STP) outlines the detailed testing strategy and framework for the **Ambulo Hiking App**, a cross-platform application developed using the Flutter framework. Ambulo is designed to enhance the hiking experience by offering a unified platform that integrates personalized route planning, real-time GPS-based navigation, dynamic weather monitoring, and interactive trail sharing. It leverages both Firebase and external APIs to provide users with a rich and adaptive interface tailored to their needs, preferences, and environmental conditions. This Software Test Plan (STP) document provides an in-depth roadmap for the testing process of the Ambulo app, ensuring both functional correctness and a high-quality user experience. The document is structured to address the various levels and types of testing that will be conducted throughout the software lifecycle. These include unit testing of core components, integration testing across connected modules, system-level validations, and final acceptance testing.

Each section of this document is crafted to guide the development and QA process through a structured testing approach. It outlines what features are in scope for testing and which ones are not included in the current cycle. Details the tools and environments to be used and presents a well-defined schedule. Moreover, the document identifies risks and provides contingency strategies to ensure testing progress is not hindered by predictable issues.

The primary goal of this plan is to create a proactive quality assurance framework that will verify that all software components function correctly under varying conditions, maintain data consistency across devices, and deliver a smooth and reliable user experience.

2. Test Items

The Ambulo application is composed of multiple layers, each integrating unique functionality essential to the user's hiking experience. The following components have been identified as critical test items, based on the actual implementation across `data_manager.dart`, `firebase_services.dart`, `trail.dart`, `user.dart`, and various manual test screens:

1. Authentication & Account Lifecycle

- Firebase-based user registration, login, password reset, and account deletion
- Admin-only operations: disabling accounts, creating new admins
- Role identification via email-based admin validation

2. User Profile Management

- Firestore user document creation and synchronization

- Live preference management: theme mode, isLogin
- Profile photo upload & deletion (Firebase Storage)
- Stat tracking: total kilometers, elevation, completed hikes

3. Trail Management System

- Creation and editing of trail documents with nested metadata
- GPX integration for route data, including alerts and waypoints
- Media gallery handling (image upload/delete)
- Trail deletion with automated cleanup across user accounts

4. Saved Hikes & History

- Saving trails locally to the user profile (with metadata)
- Adding/removing entries from hiking history
- Full retrieval of saved/completed trails with Firestore.

5. Media Operations

- Uploading trail and profile photos with Firebase Storage
- Error handling for image retrieval and deletion

6. Weather Service Integration

- Forecast queries via latitude/longitude to a third-party API
- Display of weekly forecasts and forecast summaries

7. UI Testing Hooks

- Direct access test pages: [UserTestsPage](#), [TrailTestsPage](#), [WeatherTestsPage](#)
- Manual scenario simulation (trail creation, rating, deletion, profile image tests)

3. Features to be Tested

The testing process will validate the functionality and robustness of features across all layers of the app:

Authentication & Authorization

- User sign-up, login, logout, and deletion
- Password reset
- Admin permission validation for restricted actions

User Profile & Preferences

- Create a user profile on registration
- Retrieve and edit preferences
- Toggle theme (light/dark)
- Profile image upload, load, and deletion

Trail Functionality

- Create/edit a trail with complete metadata (distance, time, region, etc.)
- Upload trail photos
- Append & remove alerts from GPX data
- Delete trails with the cleanup of all user references

Saved & Completed Trails

- Add/remove trails to/from the saved list
- Add/remove trails to/from completed hikes
- Retrieve all saved trail IDs

Weather Integration

- Fetch and display the current weather & forecast by coordinates
- Validate accuracy against location presets and GPS

Media Handling

- Upload trail/profile images manually and via file picker
- List, delete, and reload media assets

UI Navigation & Flow

- Page transitions: login/register → home → map/trail
- Theme persistence between sessions
- Testing TrailPage with injected trail ID and user

4. Features Not to be Tested

The following areas are explicitly out-of-scope for the current testing cycle, either because they are not **implemented yet or considered non-critical at this stage**:

- Web platform responsiveness and compatibility
- Push notifications via Firebase Cloud Messaging
- Payment systems (e.g., for premium trials or features)
- Advanced accessibility support (e.g., screen readers)
- Multi-language localization (only English preference tested)

5. Testing Strategy

Ambulo's development followed a **Test-Driven Development (TDD)** methodology, structured around iterative cycles of test creation, implementation, and validation. Each core feature began with the design of a **manual test case**, followed by the implementation of the corresponding logic in the backend or UI, and was then validated immediately using dedicated test screens. This approach ensured that every critical functionality in the app was written with verification in mind from the start.

Our process followed a clear 3-step cycle:

1. **Write a manual test scenario** in a dedicated Flutter page (e.g., [UserTestsPage](#), [TrailTestsPage](#), [WeatherTestsPage](#))
2. **Implement the backend and/or frontend feature** to fulfill the scenario
3. **Validate behavior through real-time testing**, including UI rendering and Firestore/Firebase integration

After successful validation in manual test environments, **GitHub Actions CI** was used to automate unit testing and validate the build process for each new commit.

We maintained 4 dedicated Git branches:

- [main](#) – Production-ready and stable releases
- [alpha](#) – Pre-release integration staging
- [daniel](#) – Feature development branch for frontend and UI testing
- [shai](#) – Backend, data logic, and integration logic

The working flow was as follows:

- Each developer implemented and tested features on their individual branch
- Once validated manually, the feature was merged into the [alpha](#) branch
- After passing all the manual and automated tests in [alpha](#), the release was merged into [main](#)

This structured workflow enforced a rigorous QA process, minimized regression risk, and supported traceable feature evolution over time.

Testing Types Applied

- **Unit Testing**
 - Core logic in `DataManager`, `trail.dart`, `weather_api.dart`
 - Utility classes: `user_utils.dart`, `gpx_ops.dart`
- **Integration Testing**
 - Firebase authentication and Firestore operations via `FirebaseFirestoreServices`
 - Firebase Storage testing
 - UI-state coordination with persistent preferences (theme, user data)
- **System Testing**
 - Use-case flows spanning multiple screens, user actions, and backend reactions
- **Acceptance Testing**
 - Use-case validation based on real user workflows from the Ambulo documentation, including: trail creation, navigation launch, photo uploads, alert feedback
 - Manual regression testing prior to every `main` release

This hybrid approach—manual-first and CI-supported—ensured both immediate feedback during development and long-term maintainability for production deployment.

6. Test Environment

- **Frontend:** Flutter (mobile + web)
- **Backend:** Firebase (auth + Firestore + Storage), shared preferences, Flutter (mobile + web)
- **Test Devices:** Android 11+, Emulator and physical device, Chrome, Firefox

7. Responsibilities

- **Shai Odeni** – Firebase integration, `DataManager` logic, user state testing
- **Daniel Betzalel** – Navigation flow testing, manual UI/UX walkthroughs
- **Test Collaborator (TBD)** – Manual and scripted system & acceptance tests

8. Schedule

Category	Tasks Completed
Setup & Planning	Flutter recap, stand-up calendar, project Google account, folder structure, theming
Versioning & CI	GitHub automation, manual tests in Flutter, GitHub Actions CI for unit/integration
Firebase Integration	Firebase Auth, Firestore, Storage, DataManager setup, admin user support
Authentication	Login/Register pages, auth interface, auto login, admin check, reset password
UI & Navigation	NavigationPage, MapPage, ExplorPage, TrailPage, layout fixes, center map
GPX & Map Logic	GPXOps, MapOps, NavigationRecordOps, AlertManager
Weather Services	Weather API integration, forecast display, WeatherTestsPage
Trail Management	Create/Edit Trails, image upload/delete, rating, admin tools
Profile Management	Profile pages (mobile/web), theme toggle, distance/elevation updates, logout
Recommendation & UI	TrailSuggester, distance filters, menu, title interface

9. Risks and Contingencies

1. Firestore Read/Write Limits

- **Issue:** Daily quota limits (e.g., 10,000 reads/sec) may be exceeded during peak usage.
- **Impact:** Pages like trails or recommendations may load slowly or fail when accessed simultaneously by many users.

2. Firebase Authentication Load

- **Issue:** High-frequency sign-in or token refresh requests (100 req/sec).
- **Impact:** Login screen may freeze or fail for multiple users at once.

3. Realtime Listeners Overload

- **Issue:** Heavy use of listeners (for trails, alerts, groups) can increase load significantly.
- **Impact:** App may slow down, freeze, or disconnect under high concurrent usage.

4. Media Storage and Display

- **Issue:** Large images or GPX files can strain Firebase Storage and download performance.
- **Impact:** Slow loading times in Explore page or trail detail views.